

Elementary Mixing Bounds and Experiments for the Representation Technique in Subset Sum

Le Duc Minh
minh.leduc.0210@gmail.com

Abstract—Subset sum is solved on every input in about $2^{n/2}$ steps, and nobody knows whether $2^{n/3}$ (or anything below $2^{n/2}$) is possible. The representation technique does much better on random inputs, about $2^{0.291n}$, and the missing ingredient for general inputs is usually called *mixing*: the many representations of a solution must fall into many residue classes modulo a random prime. This note does not give a faster worst-case algorithm. It gives two elementary facts about mixing and one experiment. First, if the weight- w sums take few residues modulo a prime p , then the input values themselves are concentrated modulo p . Second, averaged over a window of primes, the collision energy of the representations is at most $E_\infty + Y^2 \kappa / \pi$, which is $\mathcal{O}(Y \log Y)$ when few exact collisions occur, so a random prime mixes to within a polylogarithmic factor on every input. We then run the complete pipeline—modular filter, balanced sub-solver, disjointness match—on adversarial families. Low mixing appears only on arithmetic structure, which is easy; the hard large-value and generalized-progression families mix at 0.85–0.95 and are solved at the sub-solver cost $2^{(n/2)_{(n/8)}} = 2^{0.4057n}$. We conclude that for the coefficient set $\{0, 1\}$ the obstruction is the completeness of the balanced sub-solver, not mixing, and we state the resulting open problem. The two lemmas are machine-checked in Lean.

Index Terms—subset sum, representation technique, mixing, meet-in-the-middle, worst-case analysis

I. INTRODUCTION

Suppose you have n integers and a target. Can you pick some of them so that the picked numbers add up exactly to the target? This is the *subset sum* problem. For the numbers 3, 5, 8, 13 and target 16 the answer is yes, in two ways: $3 + 13$ and $3 + 5 + 8$.

Written as an equation, given integers $a_1, \dots, a_n$ and b , we ask whether $\exists x \in \{0, 1\}^n : \sum_i a_i x_i = b$.

A. Why it matters

Subset sum is one of the standard NP-complete problems. Trying every subset costs 2^n steps; meet-in-the-middle of Horowitz and Sahni [1] brings this to about $2^{n/2}$, and that is still the best bound that holds on every input, fifty years later. Schroeppe and Shamir [2] kept the $2^{n/2}$ time with $2^{n/4}$ memory, and Chen, Jin, Randolph and Servedio [5] removed a polynomial factor, but the exponent stayed $1/2$. Whether $2^{(1/2-\varepsilon)n}$ is possible is open.

B. What is known

On *random* inputs the representation technique of Howgrave-Graham and Joux [3], refined by Becker, Coron and Joux [4], runs in about $2^{0.291n}$ steps. Its speed comes from splitting the *answer* rather than the positions: a solution has enormously many two-piece *representations*, and after filtering

them with a random prime only a small subfamily remains to be searched. The analysis assumes the representations are spread across the residue classes—they *mix*. Randolph and Węgrzycki [6] made the technique work on every input for many coefficient sets, using a “mixing dichotomy”; the plain subset sum case $\{0, 1\}$ resisted. Austrin, Kaski, Koivisto and Nederlof [7] showed that instances with a very small or very large maximum bin size are also easy, leaving a hard middle band.

C. What this note does

- It sets up the mixing condition and defines the coverage and the collision energy (Section II), with a worked example.
- It proves two elementary lemmas: few hit residues force the input values to be concentrated modulo the prime, and one residue class can be stripped and the instance recursed (Section III). Both are machine-checked in Lean.
- It proves that the average collision energy over a window of primes is $\mathcal{O}(Y \log Y)$ when few exact collisions occur, so a random prime mixes to within a polylogarithmic factor on every input (Section IV).
- It runs the full pipeline on adversarial families and finds that low mixing coincides with easy arithmetic structure, while hard families mix and run at the sub-solver cost (Section V).

Section VI states what the experiments suggest the real obstruction is. We do not claim a faster worst-case algorithm.

D. A worked example of mixing

Let the support be $A = \{3, 5, 8, 13\}$ and let $w = 2$, so the representations are the 2-subsets. Their sums are 8, 11, 16, 13, 18, 21. Modulo $p = 7$ these are 1, 4, 2, 6, 4, 0, so $s_p = 5$ classes are hit and the coverage is $5/7 \approx 0.71$. Modulo $p = 5$ they are 3, 1, 1, 3, 3, 1, so only $s_p = 2$ classes are hit and the coverage is 0.4. A random prime rarely looks like $p = 5$; the point of this note is to say how rarely, and what poor coverage implies.

II. PRELIMINARIES AND THE MIXING CONDITION

Let $A = \{a_1, \dots, a_m\} \subseteq \mathbb{Z} \setminus \{0\}$ and let w with $1 \leq w \leq m - 1$. In the balanced regime $m = n/2$ is the support of a solution and $w = n/4$. Write $[m] = \{1, \dots, m\}$, and

$$\mathcal{Y} = \{y \subseteq [m] : |y| = w\}, \quad \Sigma(y) = \sum_{i \in y} a_i, \quad Y = \binom{m}{w},$$

so $\mathcal{Y}$ is the set of *representations*. For a prime p let

$$c_r = \#\{y \in \mathcal{Y} : \Sigma(y) \equiv r \pmod{p}\}, \quad E_p = \sum_r c_r^2,$$

the *collision energy*, and let $s_p = \#\{r : c_r > 0\}$ be the number of residue classes hit. The filter keeps the sums modulo p ; a representation survives in a random class with probability s_p/p , so the technique needs s_p close to p . We call s_p/p the *coverage*. Since $\sum_r c_r = Y$ and there are s_p nonzero terms, Cauchy–Schwarz gives

$$E_p \geq Y^2/s_p. \quad (1)$$

The *exact* collision energy is $E_\infty = \sum_v d_v^2$, where $d_v = \#\{y : \Sigma(y) = v\}$.

III. MIXING FORCES CONCENTRATION

The first lemma says that poor mixing is a statement about the values, not merely about their weight- w sums. All proofs are in Appendix A.

Lemma 1 (Concentration). $|(A \bmod p) - (A \bmod p)| \leq s_p^2$, and hence $|\{a_i \bmod p\}| \leq s_p^2 + 1$.

Poor mixing therefore forces additive structure. Structure is not the same as a small sumset: $A = \{p2^0, \dots, p2^{m-1}\}$ has $s_p = 1$ yet $|S(A)| = 2^m$; it is structured, and easy. The single-residue case is exact.

Lemma 2 (Contraction). *If $p \mid a_i - \rho$ for all i , then every representation sum satisfies $\Sigma(y) \equiv w\rho \pmod{p}$. Writing $a_i = \rho + pb_i$, every subset sum is $|y|\rho + p\Sigma_b(y)$, so p can be stripped from the values and the instance recursed.*

For larger s_p , the analogue is that $A \bmod p$ lies in a short generalized arithmetic progression (Kneser’s theorem); we do not pursue that step here.

IV. AVERAGE COLLISION ENERGY OVER PRIMES

The next result holds for *every* input, on average over primes.

Lemma 3 (Average energy). *Let $B = \max_i |a_i|$, let $\kappa = \lceil \log(2wB)/\log P \rceil$, and let $\pi = \#\{p \text{ prime} : P \leq p \leq 2P\}$. Then*

$$\frac{1}{\pi} \sum_{p \in [P, 2P]} E_p \leq E_\infty + \frac{Y^2 \kappa}{\pi}.$$

Take $P \approx Y$, so $\pi \approx Y/\ln Y$, and $B \leq 2^{\beta n}$ with $m, w = \Theta(n)$; then $\kappa \leq 2\beta + o(1)$. If few exact collisions occur, i.e. $E_\infty \leq Y \text{polylog}(Y)$, Lemma 3 gives an average of $\mathcal{O}(Y \log Y)$. Markov’s inequality controls the tail: for all but an $\mathcal{O}(1/\log Y)$ fraction of primes, $E_p = \mathcal{O}(Y \log^3 Y)$, so by (1)

$$s_p \geq \frac{Y^2}{E_p} = \frac{Y}{\text{polylog}(Y)}. \quad (2)$$

A representation is therefore found after $\text{polylog}(Y)$ retries, and at least half of the primes mix to coverage $Y/\text{polylog}$.

Table I checks the bound directly. The average energy is about $2Y$, the birthday value, well below the bound and below

TABLE I
AVERAGE ENERGY AND COVERAGE OVER EVERY PRIME $p \in [Y, 2Y]$, FOR A PLANTED BALANCED SUPPORT WITH SPARSE VALUES AND NO EXACT COLLISIONS.

n	Y	primes	avg. E_p	bound $Y^2 \kappa/\pi$	min. coverage
16	70	15	113	653	0.371
20	252	43	422	2954	0.325
22	462	68	778	6278	0.270

TABLE II
FULL PIPELINE ON ADVERSARIAL FAMILIES AT $n = 40$. COVERAGE IS THE FRACTION OF RESIDUES HIT BY THE PLANTED SOLUTION’S REPRESENTATIONS; WORK IS THE MEAN ENUMERATION, BASELINE $2^{0.4057n}$.

family	success	coverage	work
random $\beta = 1$	1.00	0.95	$2^{0.4057n}$
random $\beta = 1.5$	1.00	0.95	$2^{0.4057n}$
two-scale $qb_i + r_i$	1.00	0.94	$2^{0.4057n}$
rank-2 generalized AP	1.00	0.91	$2^{0.4057n}$
q -multiple	1.00	0.95	$2^{0.4057n}$
geometric (superincr.)	0.17	0.94	$> 2^{0.4057n}$
arithmetic (large step)	0.33	0.003	$> 2^{0.4057n}$

$Y \log Y$; and even the *pointwise minimum* coverage over all primes stays at 0.27–0.37, not only the average.

V. THE FULL PIPELINE ON ADVERSARIAL FAMILIES

Lemma 3 concerns all weight- w sums; the algorithm needs a representation of the actual solution. We therefore ran the complete pipeline: enumerate weight- $n/8$ subsets of each half and match by residue (the balanced sub-solver, $2^{\binom{n/2}{n/8}} = 2^{0.4057n}$ work per residue), filter by a random prime, and look for a disjoint pair of weight- $n/4$ pieces with exact sum b . Table II reports, for planted balanced solutions, the fraction solved within 16 residues, the coverage, and the enumeration work.

The hard families—large random values, two-scale weights $qb_i + r_i$, rank-2 generalized progressions, near-multiples—all mix at 0.91–0.95 and are solved at the sub-solver cost, up to a small constant. The only low-coverage family is the large-step arithmetic progression (coverage 0.003), which is additively structured and easy. Geometric (superincreasing) instances are high-coverage but the balanced enumeration happens to fail on them, and they are solved greedily in any case. No hard, poorly-mixing family was found.

We also probed the *profile* of the sub-solver. It enumerates only weight- $n/4$ pieces split $n/8$ – $n/8$ across the two halves, so it sees a solution only if the solution has such a balanced representation. Planting solutions with a controlled number of elements in the first half (Table III), the search succeeds with probability 1.00 exactly at the balanced share $n/4$ and 0.00 at every other share: on an off-balance solution it returns a wrong “no”. A single random permutation of the values repairs this only with the (small) probability that the support lands balanced.

TABLE III

PROFILE BARRIER AT $n = 40$: SUCCESS OF THE BALANCED SUB-SOLVER VERSUS THE SOLUTION'S FIRST-HALF SHARE (BALANCED IS $n/4 = 10$).

first-half share s	success
0	0.00
5	0.00
10	1.00 (balanced, $s = n/4$)

VI. WHERE THE BARRIER LIES

Combining the lemmas and the experiments: on every input a random prime mixes to within a polylog (Lemma 3), and in the tests the only low-coverage families are structured and easy. If mixing were the obstruction, the pipeline would already beat $2^{n/2}$ using the balanced sub-solver, since $0.4057 < 1/2$. It does not, so for $\{0, 1\}$ the obstruction must lie elsewhere. The experiments isolate two candidates.

(a) *The balanced sub-solver is incomplete.* The filtered class is enumerated by matching weight- $n/8$ subsets of two fixed halves, so only representations that are weight-balanced across the split are seen (Table III). An off-balance solution is never found, and the search returns a wrong “no”. Enumerating every weight profile instead costs $\sum_i \binom{n/2}{i} \binom{n/2}{n/4-i} = \binom{n}{n/4} = 2^{0.811n}$, above meet-in-the-middle; the recursion that replaces the balanced sub-solver—the four-way decomposition of [3], [4]—is where the published $0.291n$ bound is won. This is a completeness obstruction, not a mixing one.

(b) *Compatibility and exactness.* Two pieces may combine to an invalid coefficient vector (a “pseudo-solution”), and recovering a compatible pair with an exact, not merely modular, sum is a sparse orthogonal-vectors step whose cost must be kept list-linear.

The open problem suggested by the experiments is therefore:

Find a broadening of the balanced representations—overlaps, as in the four-way decomposition—such that a solution can be enumerated and recovered within $2^{(1/2-\varepsilon)n}$ for a constant $\varepsilon > 0$, or an algorithm that solves the off-balance instances below $2^{n/2}$ by other means.

Remark on a recent preprint. A 2025 preprint [8] claims a guaranteed $2^{n/2-\varepsilon}$ via a “controlled aliasing” that replaces one element by another inside a half’s partial sums and keeps one canonical subset per aliased sum. The mechanism is unsound: in the half $\{1, 2\}$, aliasing $2 \rightarrow 1$ gives the canonical subset $\{1, 2\}$ a stored sum of 2 although its true sum is 3, so the merge can return a false positive and the subset $\{2\}$ is lost. The claimed theorem is deferred to a companion paper and is not established.

VII. CONCLUSION

Mixing in the representation technique obeys an elementary, input-independent bound: on average over primes the collision energy is $\mathcal{O}(Y \log Y)$, and pointwise coverage stayed above 0.27 on every input we tested. In the full pipeline the hard,

unstructured families mix at 0.9 and run at the balanced sub-solver cost, while low mixing coincides with arithmetic structure and easy inputs. For the coefficient set $\{0, 1\}$ this points to the completeness of the balanced sub-solver, not mixing, as the obstruction: an off-balance solution is invisible to it and costs $2^{0.811n}$ to cover by brute force. Broadening the representations so that every solution is seen within $2^{(1/2-\varepsilon)n}$ is the concrete next step.

APPENDIX A PROOFS

A. Proof of Lemma 1

Fix i, j . Since $1 \leq w \leq m-1$ and $m \geq 2$, the set $[m] \setminus \{i, j\}$ has at least $m-2 \geq w-1$ elements, so there is $S \subseteq [m] \setminus \{i, j\}$ with $|S| = w-1$. The sets $S \cup \{i\}$ and $S \cup \{j\}$ both have size w , so $u = a_i + \Sigma(S)$ and $v = a_j + \Sigma(S)$ are both values of weight- w sums. Their difference is $u - v = a_i - a_j$. Letting i, j vary, every value difference modulo p is a difference of two hit residues; the hit residues number s_p , so the difference set has at most s_p^2 elements. Fixing one index as a base gives $|\{a_i \bmod p\}| \leq s_p^2 + 1$. $\square$

B. Proof of Lemma 2

If $a_i \equiv \rho \pmod{p}$ for all i , then $\Sigma(y) = \sum_{i \in y} a_i \equiv |y|\rho = w\rho \pmod{p}$. Writing $a_i = \rho + pb_i$ with $b_i = (a_i - \rho)/p \in \mathbb{Z}$, we get $\Sigma(y) = |y|\rho + p\Sigma_b(y)$, so the target determines $p\Sigma_b(y)$ given $|y|$. $\square$

C. Proof of Lemma 3

For each prime p , $E_p = \sum_r c_r^2 = \#\{(y, y') : \Sigma(y) \equiv \Sigma(y') \pmod{p}\}$, so $\sum_p E_p = \sum_{y, y'} n_{y, y'}$ with $n_{y, y'} = \#\{p \in [P, 2P] : p \mid \Sigma(y) - \Sigma(y')\}$. If $\Sigma(y) = \Sigma(y')$ then $n_{y, y'} = \pi$; there are E_∞ such ordered pairs. Otherwise $N = \Sigma(y) - \Sigma(y') \neq 0$ and $|N| \leq 2wB$ because each value is at most B in absolute value and a weight- w sum has at most w terms; the number of primes in $[P, 2P]$ dividing N is at most $\lfloor \log |N| / \log P \rfloor \leq \kappa$. The remaining Y^2 ordered pairs each contribute at most κ . Summing and dividing by π gives the claim. $\square$

REFERENCES

- [1] E. Horowitz and S. Sahni, “Computing partitions with applications to the knapsack problem,” *Journal of the ACM*, vol. 21, no. 2, pp. 277–292, 1974.
- [2] R. Schroepel and A. Shamir, “A $T = O(2^{n/2})$, $S = O(2^{n/4})$ algorithm for certain NP-complete problems,” *SIAM Journal on Computing*, vol. 10, no. 3, pp. 456–464, 1981.
- [3] N. Howgrave-Graham and A. Joux, “New generic algorithms for hard knapsacks,” in *EUROCRYPT*, 2010; IACR ePrint 2010/189.
- [4] A. Becker, J.-S. Coron, and A. Joux, “Improved generic algorithms for hard knapsacks,” in *EUROCRYPT*, 2011; IACR ePrint 2011/474.
- [5] X. Chen, Y. Jin, T. Randolph, and R. A. Servedio, “Subset sum in time $2^{n/2}/\text{poly}(n)$,” in *RANDOM*, 2023; arXiv:2301.07134.
- [6] T. Randolph and K. Węgrzycki, “Beating meet-in-the-middle for subset balancing problems,” arXiv:2511.10823, 2025.
- [7] P. Austrin, P. Kaski, M. Koivisto, and J. Nederlof, “Dense subset sum may be the hardest,” in *STACS*, 2016; arXiv:1508.06019.
- [8] J. Salas, “Beyond worst-case subset sum,” arXiv:2503.20162, 2025.